DNS Resolver Assignment Report

COMP3331/9331

Owais Wangde
z5690720

2026 Term 2

---

1 Code organisation

The submission consists of two main programs.

`parser.py` reads a single DNS message from a binary file and displays the decoded message in a readable format. It operates entirely on local files and does not perform any network communication.

Its program flow is `main()` → read message file → `print_message()` → `parse_message()`. The parser then prints the header, Questions, Answers, Authority, and Additional sections in wire order.

`resolver.py` implements the UDP DNS resolver. At startup, it reads the root hints file, binds to `127.0.0.1`, and handles each client datagram in a separate thread. Its flow is `main()` → `serve()` → `Resolver.response()`. For each query, `response()` checks the root-hints-local cases, searches the cache, and calls `resolve()` when no usable cached result is available. It then calls `build_client_response()` to send the reply.

The programs share code through the **`parser`** module. **`resolver.py`** imports `parser.py` and calls `parser.parse_message()` to decode both client queries and upstream DNS responses. It also uses the shared `Question`, `ResourceRecord`, and `DNSParseError` definitions. This keeps the parser's bounds checks, compression handling, and RDLENGTH rules the same in both programs.

The automated tests are grouped by parser, resolver-core, black-box, and integration behaviour. The integration suite includes a deterministic UDP concurrency stress test that exercises the resolver and its shared cache without depending on public DNS.

2 Data structures and algorithms

The `Question` structure stores a domain name, query type, and query class. The resource-record structure `ResourceRecord` stores:

- the owner name;
- record type and class;
- time to live (TTL);
- RDLENGTH;
- decoded record data; and
- the original RDATA bytes.

The resolver also uses `DNSRecord`, a smaller record structure for the supported DNS record types.

The DNS message representation is the header values, a list of `Question` objects, and three ordered lists of `ResourceRecord` objects for the Answer, Authority, and Additional sections. `DNSMessage` keeps the input bytes; the parsing functions pass an offset forward as each field is decoded. Names are represented as strings with a trailing dot. The encoder maps case-normalised name suffixes to packet offsets, allowing a mixed-case owner to point to an earlier name while retaining the original Question spelling.

The parser reads the DNS header, question section, and all resource records in the Answer, Authority, and Additional sections. It supports the following record types:

`A`, `NS`, `CNAME`, `MX`, and `PTR`.

The parser follows DNS compression pointers and detects pointer loops. A domain name is limited to 20 pointer jumps, and the parser verifies the end of every field before reading it. The RDLENGTH value determines the endpoint of each RDATA field. If an RDATA field is malformed, the parser reports the error while preserving the correct position for reading the following record.

The resolver begins with the root name servers and their matching A records from the root hints file. Candidate servers are tried in order. A referral uses IN-class NS records in the Authority section and matching IN-class A glue in the Additional section. When usable glue is unavailable, the resolver performs a separate lookup from the root to obtain the referred name server's A record.

Referral state is kept in a `Budget` object for one client request. It contains the deadline and counters for upstream attempts, referral levels, and CNAME records. The current candidate server list and CNAME chain are local variables in the resolution function. They are not shared between clients.

Cache entries are indexed using the following key:

`(lowercase fully qualified name, type, class)`

Each cache entry stores an expiry time calculated using `time.monotonic()`. The TTL returned to a client decreases as time passes. Records with a TTL of zero are not cached. For an alias lookup, the cache can return a complete CNAME chain followed by the requested final record set.

The cache is the only shared mutable resolver state. A `threading.Lock` protects cache reads and updates. Each UDP client request has its own worker thread, query bytes, client address and port, transaction ID, `Budget`, candidate list, and CNAME chain. This prevents one lookup from changing another lookup's resolution state.

3 Design decisions and limitations

Parsing and matching. All client and upstream packets use the same bounds-checked parser. It implements the DNS header, section layout, RDLENGTH boundaries, and compression pointers described by RFC 1035 [2, Section 4.1]. A malformed client packet receives no reply. For upstream UDP, datagrams with a wrong source endpoint, ID, opcode, or Question are unrelated and ignored, as motivated by RFC 1035 [2, Section 7.3]. Once those identity fields match, TC, malformed structure or supported RDATA, an unsupported RCODE, or a socket error fails that candidate immediately; resolution continues with the next server without waiting out the remaining timeout.

Iterative resolution. A cache miss starts from ordered root-hint addresses and sends non-recursive IPv4 UDP queries with fresh random 16-bit IDs. The answer, name-error, closer-delegation, CNAME, and server-failure states follow RFC 1034 [1, Section 5.3.3]. A referral

is accepted only when an IN-class Authority NS RRset moves to a longer relevant suffix; only matching IN-class A glue is used. Otherwise, a nested A lookup resolves the nameserver.

Aliases and negative answers. For A, NS, MX, and PTR, CNAME RRsets are kept in wire order before the final type. Targets must be consistent; loops and more than ten retained CNAME records cause SERVFAIL. A direct CNAME query returns the RRset without chasing it, consistent with RFC 1034 [1, Sections 4.3.2 and 5.2.2]. Authoritative NXDOMAIN is final, but reachable CNAME links in that same packet are retained along with links from earlier packets. AA is copied only when the complete positive, NODATA, or NXDOMAIN result came from one authoritative packet; cached and cross-packet results clear it.

Caching and response construction. Positive RRsets use lowercase `(name, type, class)` keys, absolute monotonic expiry, decreasing returned TTLs, and no TTL-zero reuse, following RFC 1035 [2, Section 6.1.3]. The response encoder preserves the client's ID, opcode, RD, and Question spelling, sets QR and RA, derives exact counts, and safely compresses case-insensitive suffix matches. At the classic UDP limit in RFC 1035 [2, Section 4.2.1], this assignment requires TC=0, so an encoding over 512 bytes becomes an empty SERVFAIL.

Limits and concurrency. Each request has a `min(30, 50 * timeout)` deadline and caps of 50 upstream attempts, ten referrals, and ten retained CNAME records. Independent worker threads prevent a slow lookup from serialising other UDP clients, targeting RFC 1035 [2, Section 6.1.1], while a lock protects the only shared mutable object, the cache. The implementation uses only Python's standard library. It does not implement TCP fallback, IPv6, EDNS(0), DNSSEC validation, negative or persistent caching, or record types outside the assignment. A production resolver would also bound worker threads and request rates.

4 Performance report

On 30 July 2026, a raw-UDP client on the same MacBook Air tested the submitted resolver and Cloudflare `1.1.1.1`. Every exchange used a fresh transaction ID, the fixed order below, and a three-second client timeout. Latency ran from `sendto` to the matching `recvfrom`; total is batch wall-clock time.

A `example.com`, `www.iana.org`, `www.python.org`, `www.unsw.edu.au`

NS `example.com`, `iana.org`, `python.org`, `unsw.edu.au`

MX `example.com`, `gmail.com`, `outlook.com`, `unsw.edu.au`

CNAME `www.python.org`, `www.wikipedia.org`, `www.microsoft.com`, `www.github.com`

PTR reversed names for `8.8.8.8`, `1.1.1.1`, `4.2.2.4`, and `8.8.4.4`

One resolver process handled cold then warm sequential batches; a fresh process handled cold then warm concurrent batches. The public batches used the same client and order, with RD set for Cloudflare. Thus only the local resolver's cache state was controlled; Cloudflare's pre-existing cache state is unknown.

Table 1: Fixed 20-query raw-UDP comparison (times in ms)

Batch	Q	OK	RC	TO	Total	Mean	Median	p95
Local cold seq.	20	18	2	0	16,865.74	843.26	301.81	2,752.44
Local warm seq.	20	18	2	0	5,924.87	296.23	0.12	2,939.55
Local cold conc.	20	18	2	0	2,847.37	825.48	334.41	2,683.31
Local warm conc.	20	18	2	0	3,132.49	358.20	69.69	2,855.03
Cloudflare seq.	20	20	0	0	927.34	46.35	11.62	168.35
Cloudflare conc.	20	20	0	0	207.32	55.86	40.13	178.41

All 120 exchanges received replies. The two local non-zero results in every batch were SERVFAIL for `www.iana.org A` and `1.1.1.1.in-addr.arpa PTR`; they were not timeouts. Warm sequential median fell from 301.81 ms to 0.12 ms because many positive answers were cached. Concurrency reduced cold batch wall time from 16.87 s to 2.85 s. Warm concurrent median also improved, although its total was slightly above cold concurrent due to Internet variation and uncached/NODATA work. Cloudflare was faster, but it is a distributed recursive service with unknown cache state, so this is contextual rather than a controlled implementation-to-implementation comparison.

Concurrency stress test

On 30 July 2026, the concurrency stress test in `tests/test_resolver_integration.py` was run on the same host. It uses a local scripted authoritative UDP server, so the result is repeatable and independent of Internet conditions. The test sends five waves of 40 clients released simultaneously: one cold-cache wave and four warm-cache waves, for 200 requests in total. Each request has a unique transaction ID, while the 40 names and expected addresses repeat across waves. The upstream server adds deterministic delays of 0–21 ms to reorder cold responses.

Table 2: Concurrent resolver stress-test result

Measure	**Result**
Client requests attempted	200
Responses received and validated	200
Timeouts or client errors	0
Transaction-ID or answer mismatches	0
Upstream queries	40

The 40 upstream queries came from the cold wave; all 160 later requests were served from the shared cache. The assertions verify every response ID and A record rather than only counting replies. This stresses simultaneous socket handling, per-request state isolation, concurrent cache writes, and concurrent cache reads. The existing slow-versus-fast test separately confirms that a 300 ms upstream delay does not serialise an unrelated request.

5 Security discussion

DNS cache poisoning. An attacker may send a forged UDP response before the legitimate upstream response arrives.

For each upstream request, this resolver generates a new random transaction ID. It also validates the response's source IP address, source port, transaction ID, flags, and Question section. These checks reduce the likelihood that a forged response will be accepted. However, they do not provide cryptographic authentication. DNSSEC validation is outside the scope of the assignment.

Resource exhaustion and amplification. A slow upstream server can occupy a worker thread, while many client packets can create threads. A resolver that is reachable from the Internet can also be used to reflect larger DNS responses towards another host.

The resolver limits the resources consumed by a single request through timeouts and limits on upstream attempts, referrals, and CNAME records. It also rejects malformed domain names and malformed RDATA fields safely.

Finally, the resolver binds only to `127.0.0.1`. This prevents remote Internet clients from using it as an open recursive resolver or as part of a DNS amplification attack.

6 References

[1] P. Mockapetris, *Domain Names: Concepts and Facilities*, RFC 1034, November 1987. https://datatracker.ietf.org/doc/html/rfc1034

[2] P. Mockapetris, *Domain Names: Implementation and Specification*, RFC 1035, November 1987. https://datatracker.ietf.org/doc/html/rfc1035

[3] Internet Assigned Numbers Authority, *Domain Name System (DNS) Parameters*. https://www.iana.org/assignments/dns-parameters/dns-parameters.xhtml

[4] COMP3331/9331, *Assignment: Building a DNS Resolver*, 2026 Term 2, version v1.

[5] R. Elz and R. Bush, *Clarifications to the DNS Specification*, RFC 2181, July 1997. https://datatracker.ietf.org/doc/html/rfc2181

[6] D. Andrews, *Negative Caching of DNS Queries (DNS NCACHE)*, RFC 2308, March 1998. https://datatracker.ietf.org/doc/html/rfc2308

[7] R. Arends et al., *DNS Security Introduction and Requirements*, RFC 4033, March 2005. https://datatracker.ietf.org/doc/html/rfc4033

7 AI interaction summary

Codex was used between 15 and 30 July 2026 as a learning, design-review, and testing aid. During 15–17 July, I used it to clarify DNS name compression, including pointer offsets, nested pointers, loop and bounds checks, RDLENGTH boundaries, and retaining the outer-field offset after following a pointer. On 16 July, AI produced a sample name-decoding algorithm, which I rewrote and adapted to the **`parser.py`** structure rather than copying directly.

From 19–26 July, I discussed how to organise **`resolver.py`** and reasoned through later assignment stages. Stage 2 discussion covered root-hints parsing, local root NS/A responses, flags, counts, and failure handling; the 22 July interaction was conceptual and generated no submitted implementation. Stage 3 discussion covered iterative referrals, IN-class glue, nested nameserver-address lookups, CNAME and negative answers, caching, timeouts, response size, and concurrency.

On 27 July, AI suggested parser, resolver, malformed-packet, cache, failover, and concurrency tests. On 30 July, it helped design the deterministic 200-request stress test, identify relevant RFC 1034/1035 sections after tutor feedback, and audit edge cases. This affected **`tests/`**, **`report.tex`**, and targeted parts of **`resolver.py`**.

I retained ideas that matched the specification, rewrote or simplified proposed algorithms, and rejected external DNS libraries, changed assessed interfaces, public recursive resolution, TCP fallback, and other out-of-scope features. I checked retained work against the assignment, RFCs, released packets, automated and live tests, repeated stress runs, and PDF inspection. I reviewed and can explain the parser, compression, iterative resolver, cache, and concurrency components in the viva. Detailed dated interactions are in **`ai-log.txt`**.